



PARADIGMAS DE LINGUAGENS DE PROGRAMAÇÃO

Programação Funcional em Python

Do paradigma à prática — estudo de caso de um sistema real em produção

Juliano Freitas

Faculdade Nova Roma

ROTEIRO

O caminho de hoje

1 · O que é e por que importa 6 · Closures e decoradores

2 · Os pilares 7 · Avaliação preguiçosa

3 · Funções como valores 8 · Recursão

4 · Ferramental do dia a dia 9 · Estudo de caso: o projeto

5 · functools 10 · PF × Imperativo e limites

Ao longo do caminho, três pausas com dicas práticas de Python.

OBJETIVOS

Ao final, você será capaz de:

Identificar

Reconhecer código funcional e seus conceitos em Python real.

Escrever

Aplicar funções puras, HOFs, closures e decoradores com propósito.

Justificar

Explicar POR QUE o estilo funcional ajuda — e onde não compensa.

A aula usa um sistema real — o projeto Ponte Italia — como fio condutor: cada conceito aparece em código que roda.

01

O que é e por que importa

Paradigmas, origem e a ideia central da programação funcional

Duas formas de dizer a mesma coisa

Imperativo — COMO fazer

```
total = 0
for n in numeros:
    total += n
```

Funcional — O QUE se quer

```
total = sum(numeros)
```

A programação funcional é um estilo declarativo: compomos funções em vez de encadear comandos que alteram variáveis.

Do cálculo- λ ao Python de hoje

1936

Cálculo- λ : Church formaliza computação só com funções.

1958

Lisp: McCarthy traz funções como dados e recursão.

1990

Haskell: puramente funcional, com avaliação preguiçosa.

Hoje

Python: multiparadigma — map, lambda, functools.

Três compromissos

1 · Funções no centro — o programa é avaliação e composição de funções.

2 · Evitar estado mutável — em vez de alterar, produzir um novo valor.

3 · Sem efeitos colaterais — a função conversa pelo retorno.

Python não é Haskell — e tudo bem

Funcional onde calcula

Regras de negócio, transformação de dados, cálculos testáveis.

Imperativo nas bordas

Ler e gravar banco, requisições HTTP, arquivos e relógio.

Python é multiparadigma. A boa engenharia não é ser 100% funcional, e sim usar o estilo funcional ONDE ele paga.

02

Os pilares

Pureza, efeitos colaterais, imutabilidade e transparência referencial

Função pura

```
# pura
def dobro(x):
    return x * 2

# impura: depende de estado externo
total = 0
def acumula(x):
    global total
    total += x      # efeito
    return total
```

Três marcas

- Mesma entrada, mesma saída, sempre.
- Não lê nem altera nada fora dela.
- Depende só dos argumentos.

O que é um efeito colateral

Mutar estado externo

Variável global, atributo, lista compartilhada.

I/O

Imprimir, ler banco, requisição de rede, arquivo.

Depender do ambiente

Hora atual, número aleatório, variável de ambiente.

Efeitos não são proibidos — são empurrados para as BORDAS. O núcleo que calcula fica puro, previsível e testável.

Não altero — crio um novo

```
from dataclasses import dataclass

@dataclass(frozen=True)
class DocumentoResumo:
    categoria: Categoria
    status: Status

d = DocumentoResumo(cat, ok)
d.status = venc    # ERRO: congelado
```

Como se faz

- tuple, frozenset, str já são imutáveis.
- frozen=True: objetos que não mudam.
- Sem mutação = sem bug de estado compartilhado.

Trocar a chamada pelo seu valor

```
preco(3) -> 15.0          # sempre  
  
# então estas duas linhas são iguais:  
total = preco(3) + preco(3)  
  
p = preco(3)  
total = p + p
```

Por que importa

- Raciocinar sobre o código como em álgebra.
- Habilita cache seguro (memoização).
- Permite paralelizar sem travas.

É a propriedade que só funções puras têm — e a base dos slides de `lru_cache` mais à frente.

calcular_status: o relógio entra pela porta

```
def calcular_status(
    validade: date | None,
    hoje: date,          # injetado!
    janela: int = 30,
) -> StatusDocumento:
    if validade is None:
        return Status.OK
    if validade < hoje:
        return Status.VENCIDO
    if validade <= hoje + dias(janela):
        return Status.VENCENDO
    return Status.OK
```

Por que é pura

- hoje é parâmetro, não `datetime.now()`.
- O status não é gravado: é derivado na leitura.
- Testável sem relógio, sem mock, sem banco.

Três ganhos que caem no seu colo

01 · Testabilidade

Um teste = três linhas. Sem fixture, sem mock, sem banco de pé.

02 · Concorrência segura

Sem estado compartilhado, dois pedidos simultâneos não colidem.

03 · Raciocínio local

A regra inteira cabe na tela; nada acontece 'à distância'.

Deixe as ferramentas vigiarem por você

- ruff (lint) + black (formatação) + mypy (tipos) rodando no CI desde o dia 1.
- No projeto os três rodam em todo push: 188 testes e zero erro de tipo.
- mypy --strict pegou um bug real que passaria despercebido em revisão.
- Custo: 10 minutos de configuração. Retorno: o resto do semestre.

03

Funções como valores

Primeira classe, funções de alta ordem e composição

Funções são dados

```
def dobro(x): return x * 2

f = dobro          # sem parênteses!
f(10)              # 20

ops = {
    "x2": dobro,
    "neg": lambda x: -x,
}
```

O que dá para fazer

- Guardar em variável, lista ou dicionário.
- Passar como argumento.
- Retornar de outra função.

Funções de alta ordem (HOF)

```
# map, filter e sorted já são HOFs:  
list(map(str.upper, nomes))  
list(filter(lambda n: n.ativo, users))  
sorted(cursos, key=lambda c: c.prazo)
```

HOF = uma função que recebe outra função como argumento e/ou devolve uma função.

O registry de parsers

```
# scraper/sources/__init__.py
PARSERS = {
    "universitaly": parser_universitaly,
    "google_news": parser_google_news,
}

# a função é escolhida pelo NOME:
universidades = PARSERS[fonte](html)
```

O ganho

- Zero if fonte == '...' espalhado.
- Fonte nova = uma linha no dicionário.
- O código de coleta nunca muda.

agrupar_por_categoria recebe a chave

```
def agrupar_por_categoria(itens, chave):  
    return {  
        cat: tuple(i for i in itens  
                    if chave(i) == cat)  
        for cat in Categoria  
    }
```

```
# a MESMA função serve documentos  
# e requisitos:  
agrupar_por_categoria(  
    docs, lambda d: d.categoria)
```

Composição

- A função de extração entra por parâmetro.
- O agrupamento não conhece o tipo concreto.
- Peças puras se encaixam: status → gap.

04

Ferramental do dia a dia

lambda, map, filter e comprehensions

Função anônima, curtinha

```
quadrado = lambda x: x ** 2
quadrado(5)      # 25

# uso típico: argumento inline
sorted(cursos, key=lambda c: c.prazo)
```

PEP 8

- lambda só inline, como argumento.
- Precisa de nome? Use def.
- Uma expressão: sem if/for de bloco.

Transformar e selecionar sem loop

map — transforma cada item

```
list(map(str.strip, linhas))  
  
# equivale a:  
[s.strip() for s in linhas]
```

filter — mantém quem passa

```
list(filter(eh_valida, urls))  
  
# equivale a:  
[u for u in urls if eh_valida(u)]
```

Em Python, a comprehension costuma ser a forma mais legível. map/filter ganham quando a função já existe e tem nome.

ordenar_por_prazo

```
def ordenar_por_prazo(cursos, prazo_de):  
    return tuple(sorted(  
        cursos,  
        key=lambda c: (  
            prazo_de(c) is None,  
            prazo_de(c) or date.max,  
        ),  
    ))
```

A regra na chave

- Quem tem prazo vem antes.
- Prazo mais próximo primeiro.
- prazo_de é injetado: serve a vários tipos.

O jeito pythônico de transformar dados

```
nomes = [u.nome for u in users if u.ativo]

por_id = {u.id: u for u in users}

unicos = {u.cidade for u in users}
```

Lê-se quase como português: 'o nome de cada usuário, para cada usuário, se ativo'.

Achatando requisitos por categoria

```
avaliacoes = tuple(
    _avaliar(req, por_cat.get(cat, ()))
    for cat, grupo in indexados.items()
    for req in grupo
)
```

Dois for aninhados e uma chamada pura por item: nenhuma lista temporária mutada, nenhum índice manual.

05

functools

reduce, partial e lru_cache — a caixa de ferramentas funcional

Dobrar uma sequência num único valor

```
from functools import reduce

total = reduce(
    lambda acc, n: acc + n.peso,
    noticias,
    0,
)
```

Como pensar

- Pense em: acumulador + item → novo acumulador.
- sum, max e any são reduces especializados.
- Comece pelo built-in; reduce para o resto.

Fixar argumentos e nascer uma função nova

```
from functools import partial

def conectar(host, porta, tls): ...

local = partial(
    conectar, "127.0.0.1", 5432)

local(tls=False) # host e porta fixos
```

A ideia

- Especializar sem herança e sem classe.
- A nova função é a antiga com parte dos
- argumentos pré-preenchidos.

Um parser por fonte, sem duplicar código

```
from functools import partial

parser_universitaly = partial(
    parse_universidades,    # genérico
    fonte="universitaly",
    seletores=SELETORES,
)
```

O ganho

- O parser genérico transforma HTML em tuplas.
- Cada fonte é uma aplicação parcial.
- Fonte nova não copia código.

Memoização em uma linha

```
from functools import lru_cache

@lru_cache(maxsize=256)
def fib(n):
    if n < 2: return n
    return fib(n-1) + fib(n-2)
```

A pegadinha

- Só é CORRETO memoizar função pura.
- Cachear algo que lê o banco devolve
- dado velho — vira bug de dado obsoleto.

requisitos_por_categoria: cache seguro por desenho

```
@lru_cache(maxsize=256)
def requisitos_por_categoria(reqs):
    return MappingProxyType(
        agrupar_por_categoria(
            reqs, lambda r: r.categoria))
```

Por que é seguro

- Entrada imutável e hashável (tupla).
- Função determinística, sem efeitos.
- Saída imutável (MappingProxyType).

O mesmo curso é comparado por muitos alunos: indexa uma vez, não uma vez por pedido.

Type hints não são burocracia — são refatoração segura

- Anote as assinaturas e deixe o mypy conferir o resto.
- Caso real do projeto: o mypy recusou um reduce cujo resultado ia direto para MappingProxyType — a inferência dirigida por contexto quebrava.
- A correção foi ligar o resultado a uma variável tipada antes de embrulhar.
- Sem tipos, isso só apareceria em produção, com dado estranho.

06

Closures e decoradores

Funções que capturam ambiente e funções que envolvem funções

Uma função que lembra do ambiente

```
def multiplicador(fator):  
    def aplicar(x):  
        return x * fator    # captura  
    return aplicar  
  
triplo = multiplicador(3)  
triplo(10)    # 30
```

A ideia

- A função interna 'fecha sobre' as
- variáveis de fora e as carrega consigo.
- É uma fábrica de funções configuradas.

Uma função que envolve outra função

```
def log(funcao):  
    @wraps(funcao)    # preserva nome  
    def wrapper(*args, **kwargs):  
        print("chamando", funcao.__name__)  
        return funcao(*args, **kwargs)  
    return wrapper  
  
@log  
def somar(a, b): return a + b
```

@log é açúcar para `somar = log(somar)`. O decorador adiciona comportamento transversal sem tocar na função original.

@cronometrar mede sem poluir a função

```
def cronometrar(funcao):  
    @wraps(funcao)  
    def wrapper(*args, **kwargs):  
        inicio = perf_counter()  
        try:  
            return funcao(*args, **kwargs)  
        finally: # mede até se falhar  
            dur = perf_counter() - inicio  
            logger.info("%.1f ms", dur*1000)  
    return wrapper
```

Detalhes

- @wraps mantém introspecção e docs.
- finally garante a medição mesmo com erro.
- Reusável em qualquer função.

retry_backoff: closure parametrizada

```
def retry_backoff(tentativas=3, base=0.5,
                 fator=2.0,
                 dormir=time.sleep):
    def decorador(funcao):
        @wraps(funcao)
        def wrapper(*a, **kw):
            for i in range(tentativas):
                try:
                    return funcao(*a, **kw)
                except Exception:
                    if i == tentativas-1: raise
                    dormir(base * fator**i)
            return wrapper
        return decorador
```

Closure + teste

- Cada uso captura sua própria config.
- dormir é injetado: testa o backoff
- sem dormir de verdade.

07

Avaliação preguiçosa

Generators: produzir sob demanda, gastar memória mínima

yield: produz um de cada vez

```
def contador(n):  
    i = 0  
    while i < n:  
        yield i          # pausa e devolve  
        i += 1  
  
soma = sum(x*x for x in range(10**6))  
# gera sob demanda, sem criar a lista
```

Por quê

- Memória $O(1)$: não materializa tudo.
- Trabalha com fluxos infinitos.
- islice consome só o necessário.

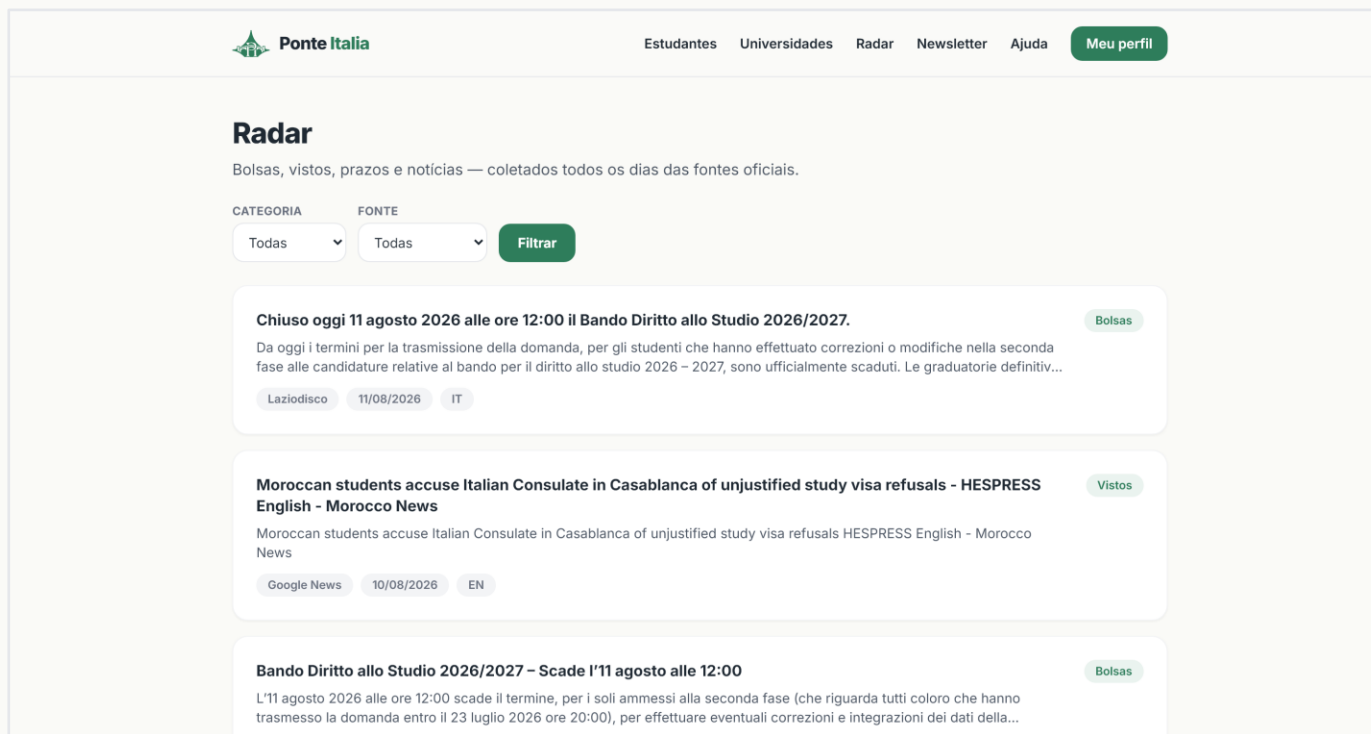
Feed de notícias item a item

```
def iterar_itens_rss(xml):  
    raiz = ElementTree.fromstring(xml)  
    for item in raiz.iterfind("channel/item"):  
        yield {...}    # um por vez  
  
def paginar(itens, pagina, por_pagina):  
    inicio = (pagina - 1) * por_pagina  
    return tuple(islice(  
        itens, inicio, inicio + por_pagina))
```

Na prática

- O Result do SQLAlchemy também é lazy.
- Um teste `pagina itertools.count()`
—
- iterável INFINITO — e termina.

O feed que os generators alimentam



111 notícias coletadas de 3 fontes. A página mostra 10: `paginar()` consome só as linhas pedidas, o resto nunca vira objeto Python.

08

Recursão

Um problema definido em termos de si mesmo

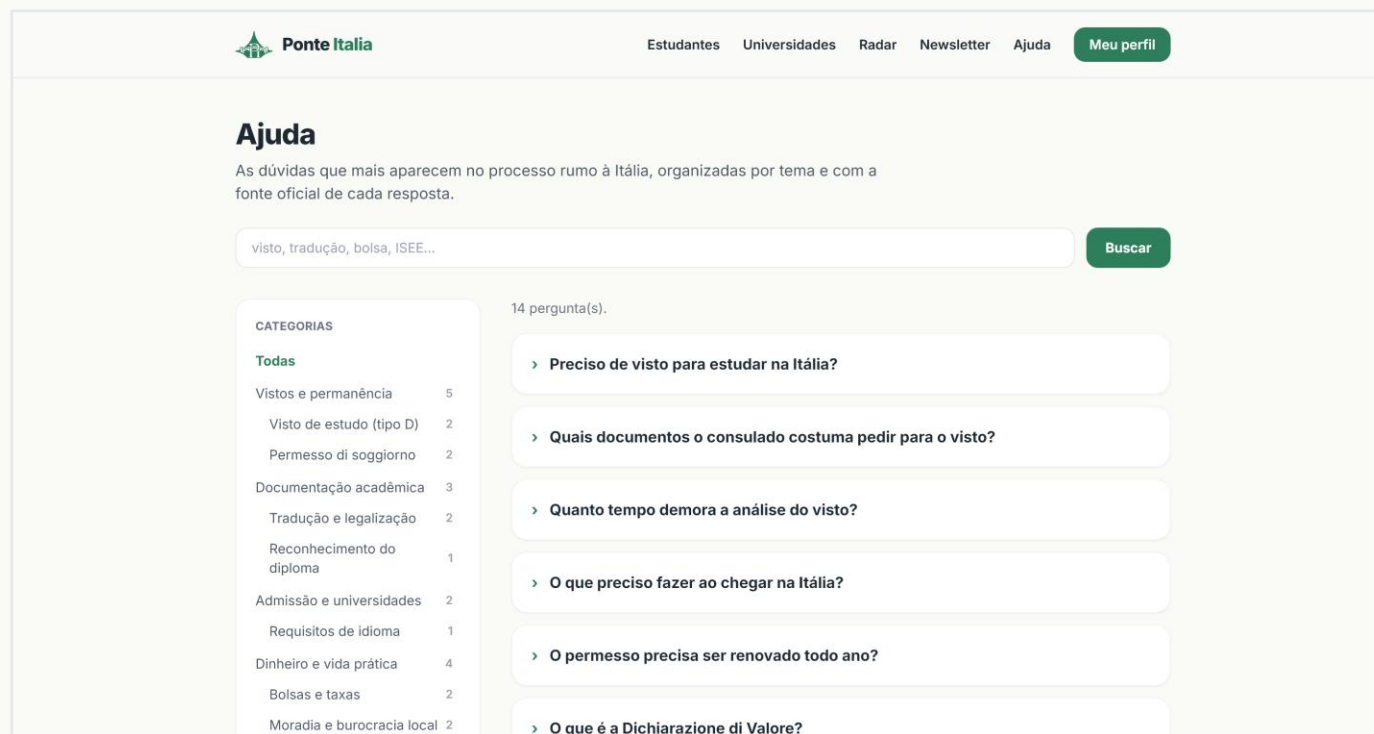
A árvore de categorias do FAQ

```
def todas_perguntas(categoria):  
    return categoria.perguntas + tuple(  
        p  
        for sub in categoria.subcategorias  
        for p in todas_perguntas(sub)  
    )  
  
def profundidade(categoria):  
    if not categoria.subcategorias:  
        return 1      # CASO BASE  
    return 1 + max(  
        profundidade(s)  
        for s in categoria.subcategorias)
```

Caso base + passo

- A estrutura é recursiva por natureza.
- Sem TCO: limite de ~1000 frames.
- O FAQ real tem 3 níveis — folga enorme.

A recursão renderizada na tela



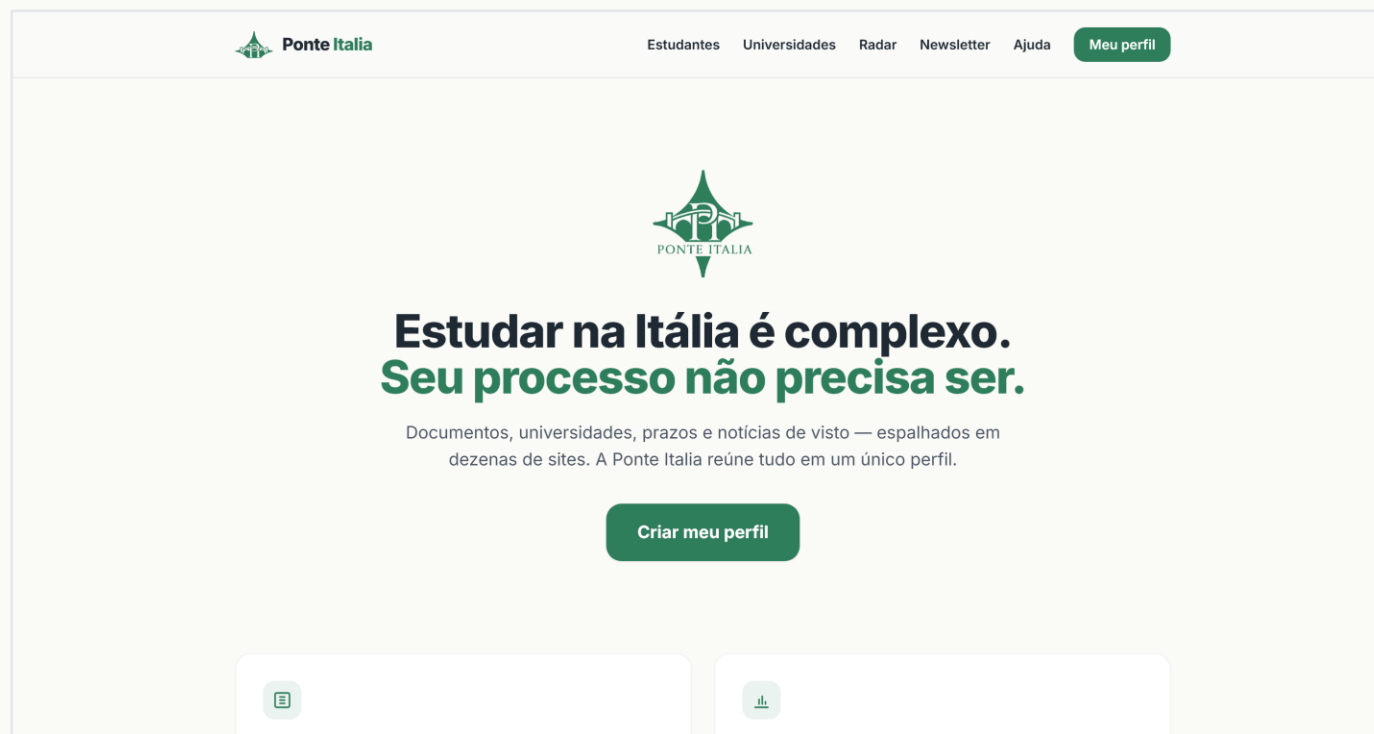
'Vistos e permanência' mostra 5 perguntas: 1 própria + 4 das subcategorias. Esse número vem de `contar_perguntas()` descendo a árvore inteira.

09

Estudo de caso: Ponte Italia

Onde todos os conceitos se encontram em código que roda

Uma plataforma real, não um exemplo de aula



FastAPI + MySQL + Redis + worker .NET. 6 sprints, 188 testes automatizados, CI com lint, tipos e testes em todo push.

Cálculo puro no centro, efeitos nas bordas

routers/

HTTP e banco — fazem o I/O e entregam dados imutáveis.

services/

NÚCLEO PURO: calcular_gap, calcular_status, curadoria, faq.

scraper/

Rede e HTML — coleta educada, isolada por fonte.

As bordas fazem I/O e entregam dados imutáveis ao núcleo; o núcleo só calcula e devolve. Testo o núcleo sem subir nada.

calcular_gap: o cálculo mais importante

```
def calcular_gap(requisitos, documentos):
    por_cat = agrupar_por_categoria(
        documentos, lambda d: d.categoria)
    aval = tuple(
        _avaliar(req, por_cat.get(cat, ()))
        for cat, grupo in indexados.items()
        for req in grupo
    )
    tri = lambda s: tuple(
        i for st, i in aval if st == s)
    return Gap(tri("atendido"),
               tri("faltando"),
               tri("vencendo"))
```

100% pura

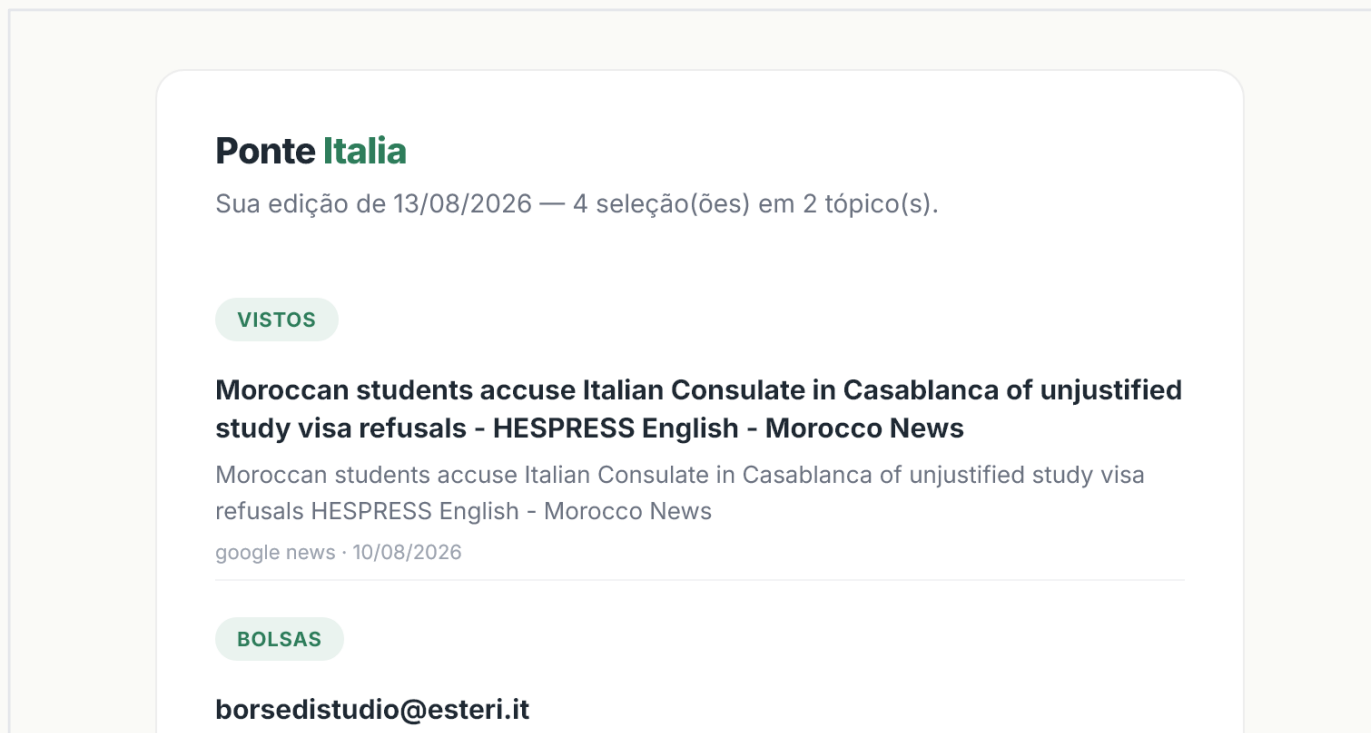
- Tuplas congeladas entram, Gap sai.
- Não conhece banco, HTTP nem relógio.
- 14 testes cobrem os cruzamentos.

O mesmo calcular_gap que roda em 14 testes

The screenshot shows a web application interface for 'Ponte Italia'. At the top, there's a navigation bar with links: 'Estudantes', 'Universidades', 'Radar', 'Newsletter', 'Ajuda', and a 'Meu perfil' button. The main content area displays the user's profile for 'Juliano Freitas', a student in 'Ciência da Computação'. Below the profile, it shows language skills: 'Italiano A2' and 'Inglês B2'. The 'Minhas universidades' section features a card for 'Artificial Intelligence' at 'Università di Bologna - Bologna'. This card includes a progress bar and three status indicators: 'Prazo próximo' (yellow), 'Interessado' (grey), and '33% pronto' (green). Below the card, there are three checklist items: 'Passaporte válido (CPF 1, CPF_2)' with a green checkmark, and 'Diploma/histórico de graduação' and 'Inglês B2 (IELTS/TOEFL)' with red X marks. At the bottom, the 'Cofre de documentos' section shows two document slots: 'CPF 1' (1.jpg) and 'CPF_2' (2.jpg), both marked 'Em dia' (on time). An 'Enviar documento' button is located to the right of the document section.

Entrada: 3 requisitos do curso + 2 documentos do cofre. Saída: um Gap congelado — 33% pronto, 1 atendido, 2 faltando. Zero consulta dentro da regra.

reduce agrupando a edição do dia



`agrupar_por_topico()` dobra as notícias das últimas 24h nos 10 tópicos, publica na fila do Redis e o worker .NET renderiza e dispara às 9h.

Cada conceito vive em código real

Função pura	<code>calcular_gap, calcular_status</code>
Imutabilidade	<code>schemas frozen, dataclasses</code>
Primeira classe / HOF	<code>registry PARSERS, agrupar_por_categoria</code>
lambda	<code>ordenar_por_prazo</code>
partial	<code>parser_universitaly</code>
lru_cache	<code>requisitos_por_categoria</code>
Decorador / closure	<code>@cronometrar, retry_backoff, @exigir_auth</code>
Generators / lazy	<code>iterar_itens_rss, paginar, janelas</code>
reduce	<code>estatisticas, agrupar_por_topico</code>
Recursão	<code>todas_perguntas, montar_arvore (FAQ)</code>

16 de 16 conceitos implementados e documentados em docs/RELATORIO_FP.md.

A armadilha do import por valor

- `from modulo import funcao` congela a REFERÊNCIA no momento do import.
- Bug real do projeto: o monkeypatch dos testes trocava `cache.get_redis`, mas a fila havia importado a função por valor — e continuou usando a original.
- Resultado: a suíte de testes escrevia no Redis de verdade, sem ninguém notar.
- Correção: importe o MÓDULO (`from app.core import cache`) e chame `cache.get_redis()`.

10

PF × Imperativo e limites

O contraste lado a lado e onde a PF não compensa

COMPARAÇÃO

O mesmo cálculo, dois mundos

Funcional — puro

```
def calcular_gap(reqs, docs):  
    ...           # só cálculo  
    return Gap(...)  
  
# testa sem banco, sem mock
```

Imperativo — acoplado

```
reqs = sessao.execute(SQL)  
for req in reqs:  
    docs = sessao.execute(SQL) # N+1  
    if doc.status == "ok":  
        # status defasado  
        atendidos.append(req)
```

A versão imperativa mistura buscar + derivar + particionar, depende de status defasado no banco e exige um MySQL de pé para cada teste.

Quando a PF NÃO compensa em Python

Cópias em excesso

Recriar grandes estruturas a cada passo custa memória. Às vezes mutar local é ok.

Recursão profunda

Sem TCO e com limite ~1000, laços são mais seguros que recursão longa.

Legibilidade

map/filter/reduce encadeados podem ficar ilegíveis — comprehension costuma vencer.

I/O é inevitável

Todo sistema real tem efeitos. O objetivo é isolá-los, não fingir que não existem.

Como aplicar isso sem exagero

Separe o cálculo do I/O: núcleo puro, efeitos nas bordas.

Dados que atravessam funções: prefira imutáveis (tuple, frozen).

Funções pequenas e nomeadas; lambda só inline.

Só memoize (lru_cache) função pura.

Deixe ruff e mypy vigiarem tipos e estilo.

PARA LEVAR

Três frases para guardar

1 · Função pura = mesma entrada, mesma saída, sem efeitos.

2 · Funções são valores: passe-as, retorne-as, componha.

3 · Isole os efeitos nas bordas; mantenha o núcleo testável.



Obrigado!

Perguntas são bem-vindas.

`github.com/julianosfreitas/scrapping\_italy`

CRÉDITOS

Fontes e recursos

- [1] Claude (Anthropic) — prompt: "gerar apresentação acadêmica de 1h20 sobre Programação Funcional em Python, usando o projeto Ponte Italia como estudo de caso, com slides de conceito, código e comparação imperativo × funcional".
- [2] Documentação oficial do Python — functools, itertools e dataclasses (docs.python.org).
- [3] PEP 8 — Guia de Estilo para Código Python (peps.python.org/pep-0008).

Código, capturas de tela e dados do projeto Ponte Italia são de autoria própria (github.com/julianosfreitas/scrapping_italy).